Week 16 — Exam Notes

Object Oriented Programming (C++)

L31: Function Templates & Class Templates **L32: Exception Handling**

L31 — Function Templates

1. What is a Function Template?

A function template is a blueprint for creating a family of functions that perform the same logical operation on different data types, without writing separate overloaded functions for each type. The compiler generates ("instantiates") the actual function code only when the template is used with a specific type.

> **Definition**
> A function template is a generic function definition that uses a placeholder type (a template parameter) instead of a fixed data type, allowing the same function body to work with int, float, char, or any user-defined type.

2. A Simple Function Template

Without templates, we would need separate overloaded functions for every type:

```
int maxVal(int a, int b) { return (a > b) ? a : b; }
float maxVal(float a, float b) { return (a > b) ? a : b; }
double maxVal(double a, double b) { return (a > b) ? a : b; }
```

With a function template, one definition handles all types:

```
template <class T>
T maxVal(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << maxVal(10, 20);       // T = int
    cout << maxVal(3.5, 2.1);     // T = double
    cout << maxVal('a', 'z');     // T = char
}
```

3. Function Template Syntax

```
template <class T>      // or: template <typename T>
returnType functionName(T param1, T param2, ...) {
    // function body using T
}
```

- **template <class T>** — the template header; declares T as a placeholder type parameter.
- **class / typename** — both keywords are interchangeable here; class does NOT mean T must be a class type.
- **T** — a symbolic name (any identifier) representing the data type supplied at call time.
- **Multiple parameters** are allowed: template <class T1, class T2>

4. What the Compiler Does

The compiler does NOT compile the template itself. Instead, when it sees a call to the template function with a particular type, it:

- Examines the argument types used in the function call.
- Substitutes the actual type in place of T throughout the template.
- Generates ("instantiates") a real, separate function for that specific type.
- Compiles that generated function normally.

This process is called template instantiation. If maxVal() is called with int, double, and char in the same program, the compiler silently creates three separate functions behind the scenes.

5. The Deciding Argument

When a function template has one parameter used to deduce T, the type of the first (or only) argument passed determines what T becomes — this is called the deciding argument.

```
template <class T>
void display(T val) {
    cout << val;
}

display(100);      // deciding argument is int -> T = int
display(5.75);     // deciding argument is double -> T = double
```

If a template has multiple parameters of type T, all arguments passed for T must resolve to the same type — the compiler uses the arguments together to deduce a single, consistent T.

6. Template Arguments Must Match

> **Rule**
>
> In a template like T func(T a, T b), both a and b must be of the SAME type when the function is called, because both are tied to the same template parameter T. Passing mismatched types causes a compiler error unless an explicit type conversion or explicit template argument is given.

```
template <class T>
T add(T a, T b) { return a + b; }

add(5, 10);        // OK -> T = int
add(5, 10.5);      // ERROR: T deduced as both int and double
add<double>(5, 10.5); // OK - explicitly force T = double
```

7. Why Not Macros?

Before templates, programmers used #define macros to write generic-looking code. Templates are strongly preferred over macros for the following reasons:

Aspect	Macros (#define)	Function Templates
Type checking	None — pure text substitution	Full type checking by the compiler
Debugging	Hard to debug (errors point to expanded text)	Easy to debug (errors point to real code)
Scope rules	No scope; can clash with other names	Respects normal C++ scope rules
Evaluation of expressions	Can cause bugs, e.g. MAX(i++, j++) evaluates twice	Arguments evaluated safely, exactly once

Aspect	Macros (#define)	Function Templates
Compile-time safety	No type safety at all	Strongly typed and safe

8. Class Templates

A class template lets an entire class (its data members and member functions) work generically with any data type, in the same way a function template generalizes a function.

```
template <class T>
class Box {
private:
    T value;
public:
    Box(T v) : value(v) {}
    T getValue() { return value; }
};

int main() {
    Box<int> intBox(10);        // T = int
    Box<string> strBox("Hi");   // T = string
    cout << intBox.getValue() << " " << strBox.getValue();
}
```

- **Instantiation:** you must specify the type in angle brackets when creating an object, e.g. Box<int>.
- **Real-life examples:** C++ Standard Library containers such as vector<T>, stack<T>, and queue<T> are all class templates.

L32 — Exception Handling

1. What is Exception Handling?

> **Definition**
> Exception handling is a mechanism in C++ that allows a program to detect and respond to runtime errors (exceptions) in a controlled way, instead of crashing abruptly. It separates normal program logic from error-handling logic.

An exception is an abnormal condition that arises during program execution, such as dividing by zero, invalid array access, or running out of memory.

2. The Three Keywords: try, catch, throw

Keyword	Purpose
try	Wraps a block of code that might raise an exception. If an exception occurs inside, control transfers to a matching catch block.
throw	Used to signal (raise) that an exception has occurred; sends an object/value to be caught.
catch	Defines a handler block that receives and processes an exception of a specific type thrown by the try block.

3. Basic Syntax

```
try {
    // code that might throw an exception
    throw someValue;
}
catch (dataType exceptionVar) {
    // code to handle the exception
}
```

Simple Example

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 0;
    try {
        if (b == 0)
            throw string("Division by zero!");
        cout << a / b;
    }
    catch (string &msg) {
        cout << "Error: " << msg;
    }
    cout << "\nProgram continues normally.";
}
```

Output:

```
Error: Division by zero!
Program continues normally.
```

4. Multiple Exception Handling (Multiple catch blocks)

A single try block can throw different types of exceptions at different times. C++ allows multiple catch blocks after one try block, and the compiler checks them in order, matching the FIRST catch block whose type matches the thrown exception.

```
#include <iostream>
using namespace std;

int main() {
    int choice = 2;
    try {
        if (choice == 1)
            throw 100;                  // int exception
        else if (choice == 2)
            throw 3.14;                 // double exception
        else
            throw string("Unknown error"); // string exception
    }
    catch (int e) {
        cout << "Caught an integer exception: " << e;
    }
    catch (double e) {
        cout << "Caught a double exception: " << e;
    }
    catch (string &e) {
        cout << "Caught a string exception: " << e;
    }
    catch (...) {
```

```
        cout << "Caught an unknown exception!";
    }
}
```

Output (choice = 2):

```
Caught a double exception: 3.14
```

- **catch (...)** — the ellipsis catch-all handler; catches ANY exception type not matched by earlier blocks. Must be the LAST catch block.
- **Order matters:** catch blocks are tested top-to-bottom; put more specific types before general/catch-all ones.

5. Real-Life Analogy

Real-Life Example

Think of a bank ATM. The "try" block is the withdrawal attempt. If you request more money than your balance (an "insufficient funds" exception) or the network is down (a "connection" exception), the ATM does not simply crash — it "throws" the specific error, and a corresponding "catch" routine displays the right message and safely returns your card, letting the ATM continue serving the next customer.

Mapped to code:

```
class InsufficientFunds {};
class NetworkError {};

void withdraw(double balance, double amount, bool networkUp) {
    try {
        if (!networkUp)
            throw NetworkError();
        if (amount > balance)
            throw InsufficientFunds();
        cout << "Dispensing cash: " << amount;
    }
    catch (InsufficientFunds &e) {
        cout << "Transaction failed: insufficient funds.";
    }
    catch (NetworkError &e) {
        cout << "Transaction failed: network error.";
    }
}
```

6. Key OOP Concepts & Definitions (Quick Reference)

Term	Definition
Exception	An object or value representing an abnormal/error condition raised during program execution.
try block	A block of code monitored for exceptions; if one occurs, execution jumps to a matching catch.
throw statement	Explicitly raises/signals an exception, optionally carrying data (an object, string, int, etc.).
catch block	A handler that matches a thrown exception's type and processes/recovers from it.
Catch-all handler	catch (...) — handles any exception type; used as a safety net, placed last.

Term	Definition
Stack unwinding	The process of destroying local objects in scope as the exception propagates from the throw point up to a matching catch.
Exception propagation	If no matching catch exists in the current function, the exception passes up to the calling function, and so on, until caught or the program terminates.
Rethrow	Using throw; (no operand) inside a catch block to pass the same exception further up the call chain.
User-defined exception	A custom class (often derived from std::exception) created to represent application-specific error conditions.

7. Why Use Exception Handling? (Benefits)

- Separates error-handling code from normal program logic — improves readability.
- Prevents abrupt program crashes; allows graceful recovery or clean shutdown.
- Allows different error types to be handled differently via multiple catch blocks.
- Supports propagation of errors across function calls without manual error-code checking at every level.
- Encourages robust, professional-quality software design.

8. Exam-Style Quick Recap

- **Function template keyword:** template <class T> or template <typename T>
- **Template instantiation:** compiler generates a type-specific function/class only when it is actually used with a type.
- **Deciding argument:** the call argument(s) whose type determines what T becomes.
- **Templates > Macros:** type safety, proper scoping, safe single evaluation, easier debugging.
- **Class template:** template <class T> class ClassName { ... }; — instantiated as ClassName<Type> obj;
- **Exception handling keywords:** try, throw, catch.
- **Multiple catch:** matched top-to-bottom by type; catch(...) is the generic catch-all and goes last.